



FORMATION PROMETHEUS & GRAFANA

Guide stagiaire — Dernier jour, version pratique

Pratiquer : des tableaux de bord qui parlent, des alertes qui servent

Sommaire

Pratiquer : des tableaux de bord qui parlent, des alertes qui servent	2
Le programme du jour	2
Rattrapage — le lab prêt en une commande	2
Les bonnes métriques : ce que Google recommande	3
PromQL avec l'IA : la méthode	4
Rappels — Grafana	5
TP A — La boutique en quatre signaux dorés	5
TP B — Le serveur en méthode USE	8
Auto-audit de vos tableaux de bord	9
Alerter sans épuiser les équipes	10
TP C — Prometheus et Alertmanager	10
TP D — L'alerting de Grafana, et la comparaison	12
Audit : trouvez les erreurs	13
Thanos express	14
Lundi matin	15
Pour aller plus loin	15
Annexe — Requêtes de secours	15

Pratiquer : des tableaux de bord qui parlent, des alertes qui servent

Le programme du jour

Aujourd'hui, on ne code pas. L'application est livrée instrumentée, les requêtes PromQL, vous les faites écrire par l'IA et vous les vérifiez. Votre travail : **construire des tableaux de bord, alerter, et juger une configuration.**

Heure	Séquence
9h00	Rattrapage : le lab prêt en une commande
9h15	Les bonnes métriques (Google SRE) et PromQL avec l'IA
9h40	TP A — La boutique en quatre signaux dorés
10h30	<i>Pause</i>
11h00	TP A, suite et finitions
11h45	TP B — Le serveur en méthode USE
12h15	Auto-audit de vos tableaux de bord
12h30	<i>Déjeuner</i>
13h30	Alerter sans épuiser les équipes
13h40	TP C — Prometheus et Alertmanager
14h30	<i>Pause</i>
15h00	TP D — L'alerting de Grafana, et la comparaison
15h30	Audit : trouvez les erreurs
15h45	Thanos express
15h55	Lundi matin
16h00	Fin

Rattrapage — le lab prêt en une commande

Dans le terminal de votre Codespace (ou de votre machine), à la racine du dépôt :

```
git fetch origin
git checkout origin/formation-2026 -- rattrapage/
bash rattrapage/appliquer.sh
```

Le script sauvegarde vos fichiers dans `rattrapage/sauvegarde-<date>/`, active les briques 01 à 06, dépose l'application instrumentée et la configuration Prometheus complète, démarre la stack et affiche l'état des cibles. La première fois, comptez 2 à 4 minutes.

Résultat attendu : `10 / 10 cibles UP`. Sinon, relancez le script une fois ; si c'est toujours rouge, levez la main.

Les onglets à ouvrir (onglet **PORTS** du Codespace, icône globe) :

Port	Service	Identifiants
3000	Grafana	admin / formation
9090	Prometheus	
9093	Alertmanager	
8080	Inbox (les notifications arrivent ici)	
5001	shop-api-1, page <code>/metrics</code>	

Les commandes pour casser la boutique, qui serviront toute la journée :

Commande	Effet
<code>./lab.sh chaos latency on / off</code>	tout devient 10 fois plus lent
<code>./lab.sh chaos errors on / off</code>	40 % des appels API échouent
<code>./lab.sh chaos cpu 120</code>	le CPU s'emballe pendant 2 minutes
<code>./lab.sh traffic 30 / traffic 6</code>	30 clients par seconde, puis retour à la normale
<code>docker stop shop-api-2 / docker start shop-api-2</code>	une instance tombe, puis revient
<code>./lab.sh chaos reset</code>	tout remettre en ordre

Les bonnes métriques : ce que Google recommande

Quand on ne sait pas quoi mettre dans un tableau de bord, on ne part pas des métriques disponibles : on part d'une **méthode**.

Les quatre signaux dorés (Google, *Site Reliability Engineering*, chapitre 6) : si vous ne mesurez que quatre choses sur un service utilisé par des gens, mesurez celles-ci.

Signal	La question	Dans la boutique
Latence	Combien de temps pour répondre ? (le p95, pas la moyenne)	<code>http_request_duration_seconds</code>
Trafic	Combien de demandes ?	<code>http_requests_total</code> en requêtes/s
Erreurs	Quelle part échoue ?	les réponses 5xx sur le total

Signal	La question	Dans la boutique
Saturation	À quel point est-on plein ?	CPU, requêtes en cours

RED (*Rate, Errors, Duration*) : les trois premiers signaux, pour chaque microservice. **USE** (*Utilisation, Saturation, Errors*, Brendan Gregg) : pour chaque **ressource** d'une machine (CPU, mémoire, disque, réseau).

La règle : RED ou signaux dorés pour ce que vos **utilisateurs** voient, USE pour ce que vos **serveurs** vivent. On alerte sur le premier, on diagnostique avec le second.

PromQL avec l'IA : la méthode

Utilisez l'IA de votre choix. Une requête bien demandée tient en cinq lignes :

```
Je travaille avec Prometheus 3 et Grafana 13.
Voici les métriques disponibles (extrait de /metrics) :
<collez les lignes # HELP et # TYPE de la métrique concernée>
Les séries ont les labels job="shop-api" et instance.
Écris la requête PromQL pour un panel Grafana qui affiche : <ce que vous voulez, avec l'unité>.
Utilise $__rate_interval et le filtre instance=~"$instance". Réponds uniquement par la requête.
```

Toujours vérifier, en quatre points :

1. Elle s'exécute dans **Explore** sans erreur.
2. L'ordre de grandeur est plausible (la boutique reçoit environ 6 requêtes par seconde, pas 60 000).
3. Tout compteur (`_total`) est dans un `rate()` ou un `increase()` .
4. Vous cassez la boutique, et la courbe réagit dans le bon sens.

Sécurité : jamais de données de production sensibles (noms de clients, adresses internes, mots de passe) dans une IA publique.

Échauffement — les quatre signaux dans Explore 10 min

Grafana → **Explore** → source **Prometheus** → mode **Code**. Faites écrire par l'IA une requête par signal doré, **pour toute la boutique** (sans variable ici : remplacez `$__rate_interval` par `5m` et retirez le filtre `instance`). Vérifiez chacune avec les quatre points.

Signal	Valeur actuelle	La requête est-elle bonne du premier coup ?
Trafic		
Erreurs		
Latence p95		
Saturation		

Rappels — Grafana

Concepts. Data source → Panel (une requête + une visualisation + des options) → Dashboard (panels, variables, annotations, liens) → Folder (rangement et droits).

Choisir la visualisation. Évolution → Time series. Valeur actuelle → Stat. Niveau borné → Gauge. Comparer quelques valeurs → Bar gauge. Liste → Table. Répartition → Pie chart. État dans le temps → State timeline. Distribution → Heatmap.

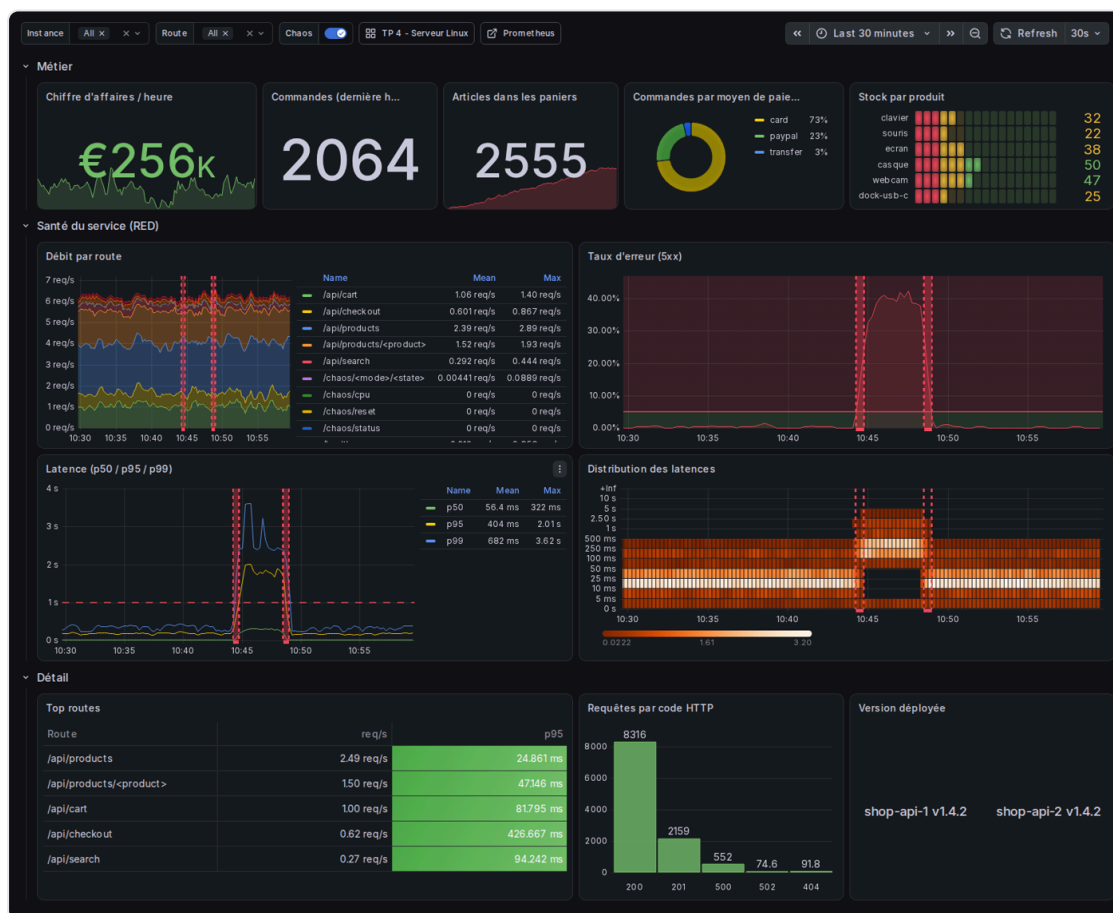
Les trois réglages qui changent tout. L'unité (*Standard options* → *Unit*), les seuils (*Thresholds*), la légende (`{{label}}`).

Variables. `$instance` dans les requêtes, avec `=~"$instance"` pour accepter plusieurs valeurs. `$__rate_interval` à la place de `[5m]` : Grafana choisit la fenêtre selon le zoom.

TP A — La boutique en quatre signaux dorés 95 min

Situation. Le directeur de la boutique en ligne veut un écran unique. En haut, en cinq secondes : « est-ce que ça va ? ». En dessous, pour l'équipe technique : quand, où, combien. Tout en bas, pour le commercial : ce qu'on vend. Le même écran doit fonctionner pour une instance ou pour toutes.

Pour chaque panel, la colonne « Demandez à l'IA » vous donne la question. Si vous êtes bloqué plus de cinq minutes sur une requête, l'annexe « Requêtes de secours » est là.



Un exemple de résultat (le vôtre sera organisé autrement, c'est normal)

Étape 0 — Le dashboard et sa variable 5 min

1. **Dashboards → New → New dashboard.**
2. **Add → Variable** (ou **Settings → Variables → New variable**) : type *Query*, nom `instance`, label `Instance`, requête `label_values(http_requests_total{job="shop-api"}, instance)`. Cochez *Multi-value* et *Include All option* (*Custom all value* : `.*`).
3. Sauvegardez : `Boutique - Signaux dorés`, dossier *Formation*. Ensuite, **Ctrl+S** (ou **Cmd+S**) toutes les dix minutes.

Étape 1 — Row « Est-ce que ça va ? » 25 min

Quatre Stat côte à côte, un par signal doré, avec une couleur de fond qui dit tout de suite si ça va. Réglages communs : *Color mode* : *Background*, *Graph mode* : *Area* (la petite courbe), requête en *Range*.

#	Panel	Demandez à l'IA	Réglages
1	Trafic	le nombre total de requêtes HTTP par seconde	Unit <i>requests/sec (rps)</i> ; couleur fixe bleue (le trafic n'est ni bon ni mauvais)
2	Erreurs	la part des réponses 5xx sur le total, entre 0 et 1	Unit <i>Percent (0.0-1.0)</i> ; seuils vert, orange à 0.01, rouge à 0.05
3	Latence p95	le 95e centile du temps de réponse, depuis l'histogramme <code>http_request_duration_seconds</code>	Unit <i>seconds (s)</i> ; seuils vert, orange à 0.5, rouge à 1
4	Saturation	le pourcentage de CPU utilisé sur la machine (Node Exporter)	Unit <i>Percent (0-100)</i> ; seuils vert, orange à 70, rouge à 90

Vérification : `./lab.sh chaos errors on`. En une à deux minutes, la case Erreurs passe à l'orange puis au rouge. `./lab.sh chaos errors off`. Puis la même chose avec `chaos latency` et la case Latence.

Le signal qui a réagi le plus lentement, et pourquoi :

Étape 2 — Row « Les signaux dans le temps » 25 min

#	Panel	Demandez à l'IA	Réglages
5	Trafic par route	les requêtes par seconde, une courbe par route	Time series ; légende <code>{{route}}</code> ; <i>Stack series</i> : <i>Normal</i> ; légende en <i>Table</i> à droite avec <i>Mean</i> et <i>Max</i>
6	Taux d'erreur	le taux d'erreur 5xx dans le temps	Time series ; <i>Percent (0.0-1.0)</i> ; seuil rouge à 0.05 avec <i>Show thresholds</i> : <i>As lines and filled regions</i> ; couleur rouge
7	Latence p50 / p95 / p99	trois requêtes, une par centile	Time series ; légendes <code>p50</code> , <code>p95</code> , <code>p99</code> ; unit <i>seconds</i> ; seuil pointillé à 1

#	Panel	Demandez à l'IA	Réglages
8	Distribution des latences	les buckets de l'histogramme, sommés par <code>1e</code> , pour une heatmap	Heatmap ; sous la requête <i>Format : Heatmap</i> ; <i>Calculate from data : No</i> ; palette <i>Oranges</i>
9	Requêtes en cours	la somme de <code>http_requests_in_progress</code>	Time series ; c'est une saturation propre à l'application

`./lab.sh chaos latency on` pendant trois minutes : regardez toute la masse de la heatmap monter.
`off`.

Étape 3 — Row « Métier » 15 min

#	Panel	Demandez à l'IA	Réglages
10	Chiffre d'affaires / heure	le chiffre d'affaires par heure, depuis le compteur <code>shop_revenue_euros_total</code>	Stat ; Unit <i>Euro (€)</i> ; 0 décimale ; couleur verte
11	Moyens de paiement	le nombre de commandes par <code>payment_method</code> sur la période affichée (<code>\$_range</code>)	Pie chart <i>Donut</i> ; Instant ; légende à droite avec <i>Percent</i>
12	Stock par produit	le stock moyen par <code>product</code> (deux instances)	Bar gauge <i>LCD</i> ; Instant ; Max 120 ; seuils rouge, orange à 20, vert à 40

Question : pourquoi le chiffre d'affaires se calcule-t-il avec `rate` alors que c'est « un montant » ?

Étape 4 — Les finitions qui font un dashboard professionnel 15 min

1. **Un panel Text** tout en haut (*Add* → *Visualization* → *Text*), trois lignes en Markdown : à quoi sert ce dashboard, qui contacter, le lien vers le runbook. C'est la première chose que lit quelqu'un qui arrive ici à 3 h du matin.
2. **Les incidents sur les courbes** : *Add* → *Annotation query*, nom `Chaos`, source Prometheus, requête `changes(shop_chaos_mode{instance=~"$instance"}[1m]) > 0`, titre `Chaos {{mode}}`, couleur rouge. Lancez un chaos : un trait rouge vertical apparaît sur **toutes** les courbes.
3. **Les liens** : *Add* → *Link*, type *Dashboard*, vers le dashboard du TP B (vous le créerez ensuite), *Keep time range* coché.
4. **La cohérence** : le même rouge veut dire la même chose partout ; aucune unité manquante.

Étape 5 — Le crash test 5 min

`./lab.sh traffic 30` : le Trafic monte, la Saturation aussi, et le chiffre d'affaires. Est-ce une panne ?
Puis `./lab.sh traffic 6`.

Ce que mon dashboard m'a dit en moins de 10 secondes :

Pour les rapides : Export → Export as code, copiez le JSON dans `grafana/dashboards/boutique-signaux-dores.json` (éditeur du Codespace), attendez 10 s, rechargez la liste des dashboards. Le même dashboard existe maintenant en code, versionnable dans Git.

TP B — Le serveur en méthode USE 30 min

Situation. Quand le haut du dashboard boutique est rouge, l'équipe infra veut descendre d'un clic vers la machine, et regarder chaque ressource sous trois angles : **Utilisation** (occupée combien de temps ?), **Saturation** (combien de travail attend ?), **Erreurs**.

Nouveau dashboard `Serveur - USE`, dossier *Formation*, variable `instance` avec la requête `label_values(node_uname_info, instance)`.

#	Ressource	Panel	Demandez à l'IA	Réglages
1	CPU	Utilisation	le % de CPU utilisé, moyenne des cœurs	Gauge ; Percent (0-100) ; seuils 70 / 90
2	CPU	Saturation	la charge sur 1 minute divisée par le nombre de cœurs	Stat ; 2 décimales ; seuils orange à 1, rouge à 2
3	CPU	Saturation (pression)	le temps d'attente CPU de <code>node_pressure_cpu_waiting_seconds_total</code> , en %	Time series ; Percent (0.0-1.0)
4	Mémoire	Utilisation	le % de mémoire utilisée avec <code>MemAvailable</code> (et pas <code>MemFree</code>)	Gauge ; Percent (0-100) ; seuils 80 / 90
5	Mémoire	Saturation	les défauts de page majeurs par seconde (<code>node_vmstat_pgmajfault</code>)	Time series
6	Disque	Utilisation	le % d'espace utilisé par point de montage, sans <code>tmpfs</code> ni <code>overlay</code>	Bar gauge ; Percent ; seuils 75 / 90
7	Disque	Saturation	le temps d'occupation des disques (<code>node_disk_io_time_seconds_total</code>), en %	Time series ; Percent (0.0-1.0)
8	Réseau	Erreurs	les erreurs réseau en réception et émission, par interface	Time series ; cachez <code>1o</code>

#	Ressource	Panel	Demandez à l'IA	Réglages
9	Cibles	Disponibilité	<code>up</code> , une ligne par cible	State timeline ; <i>Value mappings</i> 1 → UP vert, 0 → DOWN rouge

Le piège du panel 2 : l'IA propose souvent `node_load1 / count(node_cpu_seconds_total{mode="idle"})` . Essayez-la : pourquoi « No data » ? Demandez-lui de corriger en lui donnant le message.

Liez ce dashboard au dashboard Boutique (et inversement). Puis `./lab.sh chaos cpu 120` et suivez la chaîne : la case Saturation de la boutique rougit, un clic, le CPU du serveur.

Le dashboard de la communauté 5 min

Dashboards → New → Import, ID `1860` , *Load*, source Prometheus, *Import*. C'est « Node Exporter Full », le plus téléchargé de grafana.com. Lequel des deux ouvririez-vous à 3 h du matin ? Lequel pour une enquête de deux heures ?

Auto-audit de vos tableaux de bord 15 min

Notez votre dashboard Boutique, un point par ligne. Puis échangez avec votre voisin et notez le sien.

#	Critère	Le mien	Celui du voisin
1	Il répond à une question, pour un public (le titre le dit)		
2	La réponse à « ça va ? » est en haut, lisible en 5 secondes		
3	Il suit une méthode (signaux dorés, RED, USE)		
4	Chaque panel a une unité		
5	Les couleurs ont le même sens partout (rouge = il faut agir)		
6	Pas plus de 12 panels visibles d'un coup (rows repliées pour le détail)		
7	Une variable plutôt que des copies du dashboard		
8	Les incidents et les déploiements apparaissent (annotations)		

#	Critère	Le mien	Celui du voisin
9	Un lien mène au niveau suivant (le serveur, les logs, le runbook)		
10	Il est rangé dans un dossier, et exportable en code		
Total sur 10			

Le modèle de maturité de Grafana : *faible* (des copies partout, personne ne sait lequel est le bon), *moyen* (une méthode, des variables, des liens), *élevé* (tout en code, relu, le même modèle pour chaque service, on arrive sur le dashboard depuis l'alerte). Et chez vous ?

Alerter sans épuiser les équipes

Les règles de Google SRE, en quatre lignes :

- On alerte sur un **symptôme** (les clients souffrent), pas sur une **cause** (le CPU est haut).
- Toute alerte qui réveille quelqu'un doit être **urgente** et **actionnable**. Sinon, c'est un panel de dashboard.
- Trois niveaux suffisent : `critical` (on réveille), `warning` (demain matin), `info` (un ticket).
- Une alerte = un responsable, une sévérité, un runbook.

Deux systèmes d'alerte, les mêmes concepts sous d'autres noms :

Prometheus + Alertmanager	Grafana
règle dans un fichier YAML (<code>alerts.yml</code>)	<i>Alert rule</i> , dans l'interface ou en provisioning
<code>for</code>	<i>Pending period</i>
receiver	<i>Contact point</i>
route	<i>Notification policy</i>
silence	<i>Silence</i>
<code>time_intervals</code>	<i>Mute timing</i>
<code>inhibit_rules</code>	(pas d'équivalent direct)

TP C — Prometheus et Alertmanager 50 min

Situation. L'astreinte veut recevoir tout ce qui est critique, dans Teams. L'équipe boutique veut ses alertes dans son canal Slack. L'infra veut les siennes dans sa boîte. Et personne ne veut recevoir « le paiement est lent » quand le site est déjà en erreur.

Partie 1 — Les règles 10 min

```
cp rattrapage/alerting/alerts.yml prometheus/rules/alerts.yml
./lab.sh check
./lab.sh reload
```

Ouvrez `prometheus/rules/alerts.yml` dans l'éditeur du Codespace, puis Prometheus → **Alerts**.

1. Quelles alertes portent sur un **symptôme**, lesquelles sur une **cause** ?
2. À quoi sert `for: 1m` ? Et le label `team` ?
3. Laquelle réveillerait quelqu'un la nuit ? Laquelle ne devrait jamais le faire ?

Partie 2 — L'arbre de routage 15 min

```
cp rattrapage/alerting/alertmanager.yml alertmanager/alertmanager.yml
./lab.sh check
./lab.sh reload
docker compose exec alertmanager amtool config routes
```

Dessinez l'arbre sur votre guide. Puis **prédisez**, avant de vérifier, vers quels receivers part chaque alerte. Vérifiez avec `docker compose exec alertmanager amtool config routes test severity=critical team=boutique` (en changeant les labels).

Labels de l'alerte	Ma prédiction	Réponse d'amtool
<code>severity=critical team=boutique</code>		
<code>severity=warning team=boutique</code>		
<code>severity=warning team=infra</code>		
<code>severity=critical (sans team)</code>		
<code>severity=info team=logistique</code>		

Que change `continue: true` ? Retirez-le, rechargez, retestez la première ligne.

Partie 3 — Casser et suivre 10 min

1. Ouvrez côte à côte : Prometheus **Alerts**, l'Alertmanager (9093), l'Inbox (8080).
2. `./lab.sh chaos errors on`. Notez l'heure de chaque étape.
3. Dans l'Inbox : combien de messages, sur quels canaux (Teams, Slack) ? Pourquoi deux ?

4. `./lab.sh chaos errors off` : quand arrive le « resolved » ?

Pending à ... · Firing à ... · Inbox à ... · Résolue à ...

Partie 4 — L'inhibition 10 min

Si le site est en erreur, inutile d'alerter aussi sur la lenteur : une alerte critique d'une équipe fait taire les warnings de la même équipe. À la fin de `alertmanager/alertmanager.yml`, remplacez `inhibit_rules: []` par :

```
inhibit_rules:
- source_matchers:
  - severity = critical
  target_matchers:
  - severity = warning
  equal: ["team"]
```

`./lab.sh check`, `./lab.sh reload`. Puis `./lab.sh chaos errors on` et `./lab.sh chaos latency on`. Au bout de deux minutes, dans l'Alertmanager, cochez **Inhibited** : `ShopCheckoutSlow` est là, mais n'a rien envoyé. En ligne de commande : `docker compose exec alertmanager amtool alert query --inhibited`.

Partie 5 — Le silence 5 min

Une maintenance est prévue : on coupe les notifications de la boutique pendant 30 minutes. Alertmanager → **New Silence** : matcher `team="boutique"`, durée 30 min, auteur, commentaire. L'alerte existe toujours, mais ne notifie plus. Supprimez le silence (*Expire*), puis `./lab.sh chaos reset`.

TP D — L'alerting de Grafana, et la comparaison 30 min

Situation. Le directeur commercial ne lira jamais un fichier YAML. Il veut être prévenu si le chiffre d'affaires s'effondre, et pouvoir ajuster le seuil lui-même, dans Grafana. On ne duplique **pas** les alertes techniques du TP C : chacun son terrain.

Partie 1 — Deux contact points 5 min

Alerting → Notification configuration → Contact points → + Create contact point :

- `equipe-commerce` : intégration *Webhook*, URL `http://inbox:8080/webhook/commerce`. **Test**, vérifiez l'Inbox, **Save**.
- `astreinte-grafana` : intégration *Microsoft Teams*, URL `http://inbox:8080/teams/grafana`. **Test**, **Save**.

Partie 2 — La politique de notification 5 min

Notification policies :

- Default policy* → **Edit** : contact point `astreinte-grafana`.
- + **New child policy** : matcher `team = commerce`, contact point `equipe-commerce`. Dépliez *Override general timings* : *Group wait* 10s, *Group interval* 1m.

C'est l'arbre de routage du TP C, en clics.

Partie 3 — La règle métier 10 min

Alerting → Alert rules → + New alert rule :

1. Nom : `Chiffre d'affaires en chute`.
2. Requête A : le chiffre d'affaires par heure (votre panel 10 du TP A, avec `[2m]` au lieu de `$__rate_interval`), type **Instant**. Regardez la valeur actuelle dans *Preview*.
3. Condition : *IS BELOW* la **moitié** de la valeur actuelle.
4. Dossier *Formation*, groupe d'évaluation `commerce` (intervalle 1 min), *Pending period* 1m.
5. Labels : `team=commerce`, `severity=warning`.
6. *Configure notifications* : laissez **Use notification policy** (c'est la politique qui route).
7. *Summary* : Le chiffre d'affaires est tombé à `{{ humanize $values.A.Value }}` €/h. *Link dashboard and panel* : votre panel Chiffre d'affaires.
8. **Save rule and exit**. Puis `./lab.sh traffic 1` : moins de clients, moins de ventes. Suivez l'état de la règle (*Normal* → *Pending* → *Firing*) et l'Inbox, canal `commerce`. Revenez à `./lab.sh traffic 6`.

Partie 4 — La mise en sourdine 3 min

Time intervals → + Add time interval : `week-end`, jours *saturday* et *sunday*, fuseau Europe/Paris.

Appliquez-le en *Mute timings* sur la politique `team = commerce`. Le commercial ne sera plus dérangé le week-end.

Partie 5 — Comparer 7 min

Remplissez le tableau à partir de ce que vous venez de faire.

	Prometheus + Alertmanager	Grafana
Où vit la configuration ?		
Comment la versionner et la relire ?		
Comment la tester avant de la déployer ?		
Qui peut la modifier ?		
Peut-elle combiner plusieurs sources (SQL, logs) ?		
Que se passe-t-il si Grafana tombe ?		
Pour quelles alertes je l'utiliserais chez moi ?		

Audit : trouvez les erreurs 15 min

Avant de commencer, lancez Thanos pour la séquence suivante (il démarre pendant l'audit) : `bash rattrapage/thanos.sh on`.

Une équipe vous confie sa supervision : « elle marche ». Ouvrez les trois fichiers de `rattrapage/audit/` : `docker-compose.yml` , `prometheus.yml` , `alerts.yml` . Par binôme, en **8 minutes**, trouvez tout ce qui ne va pas : configuration, sécurité, alerting. Il y a au moins douze erreurs. Un point par erreur, un point de plus par correction proposée.

Fichier	Ligne	L'erreur	La correction

Puis demandez à `promtool` combien il en trouve, lui :

```
docker run --rm -v "$PWD/rattrapage/audit:/audit" --entrypoint promtool \
  quay.io/prometheus/prometheus:v3.13.3 check config /audit/prometheus.yml
```

Thanos express 10 min

Prometheus garde ses données quelques semaines, sur un seul serveur. Thanos ajoute, sans toucher aux Prometheus : une **vue globale** sur plusieurs Prometheus, la **déduplication** de deux Prometheus jumeaux, et l'**historique long** sur du stockage objet (S3, Azure, OVH).

Si vous avez lancé `bash rattrapage/thanos.sh on` , six conteneurs de plus tournent :

1. Port **10902** : le Querier de Thanos. **Stores** : combien de sources ?
2. Requête `up{job="shop-api"}` : combien de séries ? Décochez **Use Deduplication** : combien maintenant ? D'où vient le label `replica` ?
3. Grafana → Explore → source **Thanos** : la même requête. Vos dashboards peuvent pointer sur Thanos sans rien changer d'autre.
4. Pour finir : `bash rattrapage/thanos.sh off` .

Lundi matin

Choisissez **un** service de votre périmètre. Un seul. Faites-le scraper, construisez-lui une row de signaux dorés, puis **une** alerte sur un symptôme que vos utilisateurs sentiraient, avec un responsable et un runbook. C'est tout. Le reste viendra.

Mon service : ... · Sa méthode : ... · Mon alerte : ...

Pour aller plus loin

Source	Pourquoi
Google, <i>Site Reliability Engineering</i> , ch. 6 « Monitoring Distributed Systems » — sre.google/sre-book/monitoring-distributed-systems	les quatre signaux dorés, symptômes et causes
Google, <i>The Site Reliability Workbook</i> , ch. 5 « Alerting on SLOs » — sre.google/workbook/alerting-on-slos	alerter sur un budget d'erreur
Grafana, <i>Dashboard best practices</i> — grafana.com/docs	USE, RED, modèle de maturité
Brendan Gregg, <i>The USE Method</i> — brendangregg.com/usemethod.html	la méthode USE ressource par ressource
<i>Awesome Prometheus alerts</i> — samber.github.io/awesome-prometheus-alerts	des centaines de règles prêtes à adapter
Ce dépôt, guides jours 1 à 3	tout ce qu'on n'a pas eu le temps de faire : instrumentation, recording rules, droits, Teams, sauvegarde, TP Thanos complet

Annexe — Requêtes de secours

À n'utiliser qu'après avoir essayé avec l'IA. Toutes filtrent sur la variable du dashboard.

TP A — Boutique

#	Requête
1	<code>sum(rate(http_requests_total{job="shop-api", instance=~"\$instance"}[\$__rate_interval]))</code>
2, 6	<code>sum(rate(http_requests_total{job="shop-api", instance=~"\$instance", status=~"5.."}[\$__rate_interval])) / sum(rate(http_requests_total{job="shop-api", instance=~"\$instance"}[\$__rate_interval]))</code>
3	<code>histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{job="shop-api", instance=~"\$instance"}[\$__rate_interval])))</code>
4	<code>100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[\$__rate_interval])))</code>
5	<code>sum by (route) (rate(http_requests_total{job="shop-api", instance=~"\$instance"}[\$__rate_interval]))</code>

Requête

7 la requête 3 avec 0.50 , puis 0.99 à la place de 0.95

8 `sum by (le) (rate(http_request_duration_seconds_bucket{job="shop-api", instance=~"$instance"}[$__rate_interval]))`

9 `sum(http_requests_in_progress{job="shop-api", instance=~"$instance"})`

10 `sum(rate(shop_revenue_euros_total{job="shop-api", instance=~"$instance"}[$__rate_interval])) * 3600`

11 `sum by (payment_method) (increase(shop_orders_total{job="shop-api", instance=~"$instance"}[$__range]))`

12 `avg by (product) (shop_stock_units{job="shop-api", instance=~"$instance"})`

TP B — Serveur**# Requête**

1 `100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle", instance=~"$instance"}[$__rate_interval])))`

2 `node_load1{instance=~"$instance"} / on (instance) count by (instance) (node_cpu_seconds_total{mode="idle", instance=~"$instance"})`

3 `rate(node_pressure_cpu_waiting_seconds_total{instance=~"$instance"}[$__rate_interval])`

4 `100 * (1 - node_memory_MemAvailable_bytes{instance=~"$instance"} / node_memory_MemTotal_bytes{instance=~"$instance"})`

5 `rate(node_vmstat_pgmajfault{instance=~"$instance"}[$__rate_interval])`

6 `100 * (1 - node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs", instance=~"$instance"} / node_filesystem_size_bytes{fstype!~"tmpfs|overlay|squashfs", instance=~"$instance"})`

7 `rate(node_disk_io_time_seconds_total{instance=~"$instance"}[$__rate_interval])`

8 `rate(node_network_receive_errs_total{device!="lo", instance=~"$instance"}[$__rate_interval]) + rate(node_network_transmit_errs_total{device!="lo", instance=~"$instance"}[$__rate_interval])`

9 `up`

TP D — Règle Grafana : `sum(rate(shop_revenue_euros_total{job="shop-api"}[2m])) * 3600`